Ask, Don't Guess: Learning When to Clarify in Situated Instruction Following

Anonymous authors
Paper under double-blind review

Abstract

Language agents deployed in situated environments must decide not only what a user instruction might mean, but whether it is useful and safe to act on that interpretation. Existing ambiguity and clarification evaluations often frame this as a static language-understanding problem: detect ambiguity and ask a question. We argue that clarification is instead a situated decision under uncertainty, interaction cost, and task consequences. We introduce CLARIFY-TO-ACT, a procedurally generated benchmark in which an agent observes a structured scene and a user instruction, then either acts immediately or asks one clarifying question before acting. The environment provides deterministic rewards based on final task success, clarification cost, and wrong-action cost, enabling cheap learning and evaluation from interaction without human labels or model weight updates. In a 100-episode stratified API evaluation with GPT-4.1-mini, a prompted ask-when-needed baseline reaches 0.632 net utility and misses 58.3% of oracle clarification cases. An expected-communicative-utility policy reaches 0.976 net utility with 100% task success and no missed or unnecessary clarifications. A current-model sweep on GPT-5.4-mini and GPT-5.5 preserves this zero ask-error ECUbehavior; stronger prompting narrows the gap, but plain Ask-Needed remains below ECU. Across offline diagnostics, stress tests, component ablations, and cost sensitivity analyses, the main lesson is consistent: pragmatic clarification is better modeled as value-of-information under situated action consequences than as ambiguity detection alone.

1 Introduction

A situated language agent often receives instructions that are easy to understand linguistically but underspecified for action. A household assistant asked to "bring the red mug" may see two red mugs. A file-management agent asked to "delete the old draft" may see two plausible drafts with very different consequences. A collaborative robot asked to "move a spare chair" may see several chairs, any of which would satisfy the request. These examples require more than mapping language to a likely denotation. The agent must decide whether to act now, or whether the expected value of asking the user a clarification question outweighs the cost of interrupting.

This decision is pragmatic in the classic sense: utterance interpretation depends on context, mutual knowledge, informativeness, and the agent's anticipated effect on the listener or environment (Grice, 1975; Clark & Wilkes-Gibbs, 1986; Frank & Goodman, 2012). It is also central to embodied and situated instruction following, where the same string can require different behavior depending on scene state, affordances, and task stakes (Bisk et al., 2020; Min et al., 2024). Yet many current clarification benchmarks evaluate whether a query is ambiguous in isolation (Kuhn et al., 2022; Zhang et al., 2024). This is useful, but incomplete. Some ambiguous instructions are resolved by the environment; asking is unnecessary. Some underspecified instructions are harmless because all candidate actions are outcome-equivalent. Other cases are high risk: even a probable interpretation may be unacceptable if the wrong action has high cost.

We therefore study clarification as a situated ask-or-act decision. The relevant question is not simply "is the instruction ambiguous?" but "does clarification improve expected task utility?" This distinction matters for both scientific evaluation and practical agents. An over-eager assistant that asks whenever multiple interpretations exist becomes frustrating and inefficient. A guessing assistant that rarely asks may achieve high apparent success on low-stakes cases while failing exactly when clarification is most valuable. A useful benchmark must penalize both failure modes.

CLARIFY-TO-ACTisolates this decision in a lightweight, deterministic interaction environment. Each episode contains a structured scene, a natural-language instruction, candidate user intents with prior probabilities, a hidden user intent, and task costs. The agent may either act immediately or ask one clarification question. If it asks, a simulated user returns an answer generated from the hidden intent, after which the agent must act. The final score is task reward minus interaction cost and wrong-action penalties. Because all labels and rewards are computed from the environment, the benchmark supports cheap offline analysis and API-based evaluation without collecting new human annotations.

The benchmark is intentionally small and controlled. It is not meant to replace realistic simulators or human-in-the-loop studies. Instead, it provides a diagnostic testbed for a narrow but important capability: deciding when communication is worth the cost. The design lets us create matched contrasts where surface ambiguity is held constant but the correct ask-or-act decision changes because of context, equivalence, or stakes. This allows us to test whether a policy has learned a utility-calibrated interaction rule rather than a generic uncertainty heuristic.

Contributions. This paper makes four contributions. First, we introduce CLARIFY-TO-ACT, a 1,400-episode procedural benchmark for situated clarification decisions, with balanced diagnostic categories and deterministic rewards. Second, we formalize clarification as expected communicative utility: an oracle asks only when the value of information exceeds interaction cost. Third, we evaluate deterministic baselines, a lightweight interaction-supervised controller, and API-based language-agent policies. A prompted ask-when-needed baseline improves over direct action but remains poorly calibrated; an ECUpolicy substantially improves net utility by reducing both unsafe guessing and unnecessary clarification. Fourth, we provide analyses that explain where the gains come from: ambiguity-only policies over-ask, uncertainty-only controllers miss value-relevant structure, and equivalence/context guards are necessary to prevent model-generated candidate interpretations from producing unsafe ask-or-act decisions.

2 Clarify-to-Act

CLARIFY-TO-ACTis a two-turn situated interaction benchmark. Each episode is generated from a compact scene grammar and contains a hidden intent sampled from candidate interpretations. The agent observes only the scene, the user instruction, and task metadata exposed by the prompt. It does not observe the hidden intent. The agent outputs either:

$$\{\texttt{type} : \texttt{ACT}, \texttt{action} : a, \texttt{target\_id} : o\}$$

or:

$$\{\texttt{type} : \texttt{ASK}, \texttt{question} : q\}.$$

If the agent asks, the simulated user returns one answer associated with the hidden target, such as a state, location, or ownership description, and the agent then chooses a final action. Episodes are capped at one question to focus the evaluation on the first-turn ask-or-act decision.

2.1 Diagnostic categories

Table 1 summarizes the five diagnostic categories. The canonical dataset contains 1,400 episodes: 600 train, 200 development, 400 test, and 200 OOD test episodes. Splits are balanced across categories and the overall oracle ask rate is 0.50. The OOD split keeps the same diagnostic factors but shifts object types where supported by the generator.

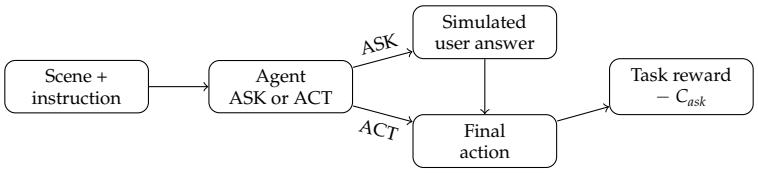

Figure 1: CLARIFY-TO-ACTinteraction protocol. The agent either acts immediately or asks one clarifying question, then receives deterministic task reward minus interaction cost.

Category	Example instruction	Oracle ask	Diagnostic contrast
Referential	Bring the black box	1.00	Matching objects, wrong target matters
Context-resolved	Pass the folder I'm using	0.00	Context or salience resolves intent
Equivalent outcome	Move a spare folder	0.00	Any target in success class works
Risk-sensitive	Delete the old draft	1.00	Wrong action has high cost
Preference/social	Put my cup away	0.50	Owner visible vs. hidden

Table 1: Benchmark categories. Each category has 280 episodes in the canonical dataset; the oracle ask rate follows expected communicative utility rather than surface ambiguity.

The categories are designed to dissociate language ambiguity from clarification utility. Referential cases require asking because multiple matching objects correspond to different success classes. Context-resolved cases contain multiple candidates, but scene salience, reachability, or active workspace context makes one interpretation utility-dominant. Equivalent-outcome cases are intentionally ambiguous at the target level, but all candidate targets share a success class; asking wastes interaction cost. Risk-sensitive cases test whether agents ask even when one candidate has high prior probability, because the wrong-action penalty is large. Preference/social cases test whether ownership or user identity is visible enough to resolve indexicals such as "my."

2.2 Reward, oracle, and evaluation metrics

The reward for an episode is

$$R = \mathbb{1}[\text{success}] - C_{ask}\mathbb{1}[\text{asked}] - C_{wrong}\mathbb{1}[\text{wrong}].$$

Success is defined by matching the hidden intent's success-equivalence class, so interchangeable actions can succeed without arbitrary target identity matches. This is important for avoiding a benchmark artifact: an agent should not be forced to ask which spare folder to move if any spare folder satisfies the instruction.

Let $\mathcal{C}$ be the set of success-equivalence classes induced by candidate intents, and let $p(c)$ be the summed prior probability of class c. If the agent acts immediately and chooses the best class, its expected utility is

$$EU(act) = p_{\max} - (1 - p_{\max})C_{wrong}, \qquad p_{\max} = \max_{c \in \mathcal{C}} p(c).$$

If the agent asks, the simulated answer is assumed to identify the hidden success class, so

$$EU(ask) = 1 - C_{ask}.$$

The oracle asks when $EU(ask) > EU(act) + \epsilon$. We use net utility as the primary metric and additionally report task success, ask rate, missed-clarification rate, and unnecessary-clarification rate. A missed clarification is an oracle-ask episode where the agent acts immediately. An unnecessary clarification is an oracle-act episode where the agent asks.

2.3 Simulated user and leakage controls

The simulated user is deterministic: each hidden target has answer templates based on visible state, location, and ownership. This keeps the task cheap and reproducible while still requiring the agent to ground the answer before acting. API smoke tests exposed possible

leakage in the preference/social category, where hidden owner names could accidentally reveal the answer. The final benchmark therefore uses neutral target identifiers, redacts hidden object owners from API prompts, gives hidden-owner objects neutral state labels, and includes `current_user` only as the user identity. Visible-owner cases remain resolvable; hidden-owner cases require clarification. A simulated-user audit finds that the deterministic answers resolve the hidden success class in 1233/1233 generated oracle-ask cases and 184/184 actual API asked rows.

3 Methods

We compare policies that differ in how they decide whether clarification is worth asking. The key comparison is not whether a method can detect that the instruction has multiple possible readings; every canonical test episode has multiple candidate interpretations. Instead, the question is whether a method conditions its ask-or-act decision on expected task utility.

DirectAct. DirectAct never asks. Offline, it chooses the highest-prior success class. In the API setting, the model receives the scene and instruction and must return an action directly. This baseline measures the cost of guessing.

AskAlways and raw ambiguity. AskAlways always asks one question, then acts from the simulated answer. Raw ambiguity asks whenever multiple candidate targets exist. Because the canonical dataset is surface-ambiguous by construction, raw ambiguity behaves like AskAlways on the main test split. These baselines measure over-clarification.

Prompted Ask-Needed. Prompted Ask-Needed is the strongest simple API baseline. The model is instructed to ask only when acting now could plausibly fail, and not to ask when context resolves the instruction or multiple choices are equivalent for task success. This baseline approximates a direct prompting strategy for pragmatic clarification and is related to recent work on implicit user-intention understanding in language-agent settings (Qian et al., 2024).

Private-reasoning Ask-Needed. A private-reasoning variant uses a prompt that encourages the model to reason before returning the same JSON schema. This tests whether additional deliberation alone calibrates the ask-or-act decision.

Expected communicative utility. The API ECUpolicy decomposes the interaction into two stages. First, the language model proposes candidate interpretations with target IDs, actions, probabilities, a context-resolution flag, an equivalence flag, and a risk estimate. Second, deterministic Python code computes expected utility from the model-proposed candidates, the ask cost, and the wrong-action cost. If $EU(ask) - EU(act)$ exceeds a margin, the model generates one clarifying question; otherwise the agent acts on the highest-probability success class.

This decomposition is deliberately conservative. The model is used as a semantic parser and candidate generator, not as the final judge of whether to ask. The final API ECUpolicy uses an ask margin of 0.075, visibility redaction for hidden-owner cases, and a conservative equivalence guard: model-declared equivalence collapses candidates only when the instruction or scene contains cues such as "spare," "any," "one of," or all candidate objects are marked spare. Without this guard, a model can incorrectly collapse distinct referents into one success class.

Learned controller. The offline learned controller is a lightweight logistic ask/act model trained from automatic oracle labels. It uses features such as number of candidates, number of success classes, top prior, entropy, ask cost, wrong-action cost, salience gap, risk, context resolution, equivalence, and expected-utility margin. It is not a learned language model; rather, it tests whether interaction-derived oracle labels are sufficient to train a transparent ask-or-act controller.

Method	Ask/act signal	Diagnostic role
DirectAct	Never asks	Guessing under ambiguity and risk
AskAlways / raw ambiguity	Always ask / target count	Over-clarification and ambiguity-only behavior
Prompted Ask-Needed	Ask-or-act prompt	Strong simple prompting baseline
CoT Ask-Needed	Private-reasoning prompt	Reasoning-only calibration test
API ECU	Model candidates + expected utility	Main utility-calibrated policy
Threshold / controller	Tuned margin / logistic model	Interaction-supervised calibration

Table 2: Method summary. The central comparison is whether the ask/act decision is calibrated to expected task utility.

4 Experiments

We organize the experiments around four questions. **RQ1:** Does utility-calibrated clarification improve task utility over direct action and prompted clarification? **RQ2:** Is surface ambiguity or uncertainty sufficient to predict when to ask? **RQ3:** Which components of the ECUpolicy matter? **RQ4:** Are the conclusions stable under category shifts, paraphrases, answer-style shifts, model changes, and cost changes?

Offline evaluation. Offline evaluation uses the full 400-episode test split and 200-episode OOD split. Because these policies have access to benchmark candidate priors and features, they should be interpreted as diagnostic upper bounds and transparent controllers rather than deployed LLM agents.

API evaluation. The main API evaluation uses GPT-4.1-mini on a 100-episode stratified subset with 20 episodes per category. The policies are DirectAct, Ask-Needed, private-reasoning Ask-Needed, and API ECU. Calls are cached; the final main cache contains 914 responses, 271,208 input tokens, and 49,117 output tokens. The final API evaluation can be replayed in cache-only mode, which fails on any cache miss rather than making a network call.

Current-model sweep. To test whether the result is specific to the historical GPT-4.1-mini run, we rerun the same 100-episode stratified API evaluation on GPT-5.4-mini and GPT-5.5. The sweep uses the same prompts, policies, episode order, and cache-only replay discipline. For GPT-5-family calls, we use the Responses API with reasoning effort set to `none`; this keeps the evaluation focused on ask-or-act prompting and candidate extraction rather than hidden deliberation budget.

Stress and robustness tests. We construct a 50-episode style-stress set by paraphrasing test instructions and shifting simulated user answers to terser location/owner-style replies. We also run a 25-episode GPT-4.1-nano sanity check, a no-API held-out ambiguity-mix diagnostic, component ablations on cached API interpretations, cost-sensitivity analyses, and a CLAMBER external sanity check (Zhang et al., 2024). Statistical comparisons use paired bootstrap confidence intervals because policies are evaluated on the same episodes.

5 Results

5.1 Main API result

Table 3 and Figure 2 show the main API result. DirectAct fails because it never asks, producing low success in referential and risk-sensitive cases. Prompted Ask-Needed improves over DirectAct but remains poorly calibrated: it misses 58.3% of oracle clarification cases and asks unnecessarily in 32.7% of oracle-act cases. The private-reasoning variant does not improve net utility (0.632) and still misses 60.4% of oracle clarification cases. API ECUreaches 0.976 net utility and 100% success, with no missed or unnecessary clarifications in this stratified evaluation.

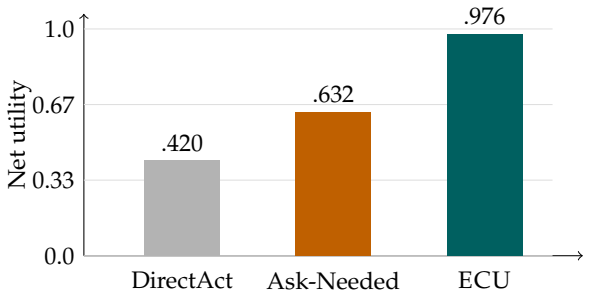

Figure 2: Main API result on 100 stratified test episodes. ECUsubstantially improves net utility over direct action and prompted ask-when-needed.

Method	N	Utility	Success	Ask	Missed	Unnec.
DirectAct	100	0.420	0.770	0.000	1.000	0.000
Ask-Needed	100	0.632	0.880	0.370	0.583	0.327
API ECU	100	0.976	1.000	0.480	0.000	0.000

Table 3: API evaluation with GPT-4.1-mini. Utility is the primary metric; missed and unnecessary clarification identify opposite calibration failures.

Because all policies run on the same episodes, paired deltas are the appropriate comparison. API ECUexceeds Ask-Needed by 0.343 net utility with 95% paired CI [0.168, 0.559], and exceeds the private-reasoning variant by 0.344 [0.171, 0.564]. Conditional on asking, both Ask-Needed and ECUsuccessfully ground the simulated answer in the main set; the gap is ask timing. ECUhas ask precision/recall/post-answer success of 1.000/1.000/1.000, while Ask-Needed has 0.541/0.417/1.000.

Table 4 asks whether model scaling solves the first-turn clarification decision. It helps, but does not remove the calibration issue. On GPT-5.4-mini, ECUreaches 0.976 utility versus 0.868 for plain Ask-Needed and 0.864 for the private-reasoning variant. On GPT-5.5, the private-reasoning prompt closes the gap and ties ECUat 0.976, but plain Ask-Needed remains lower at 0.821 because it still under-asks some oracle-clarification cases. Across all three 100-episode model rows, ECUhas zero missed and zero unnecessary clarifications.

5.2 Category-level behavior

Table 5 shows that the ECUadvantage is not driven by a single category. Prompted Ask-Needed performs well on referential ambiguity but over-asks equivalent-outcome cases and misses most risk-sensitive cases. In risk-sensitive episodes, Ask-Needed asks on only 15% of examples and obtains slightly negative utility. ECUasks in all referential and risk-sensitive cases, avoids all context-resolved and equivalent-outcome cases, and matches the oracle ask rate in preference/social cases. A no-API subset-stability diagnostic confirms that the improvement remains positive after omitting any one category (minimum +0.190) or any one episode (minimum +0.307), with a category-stratified 95% CI [0.183, 0.541].

5.3 Offline controllers and ambiguity-only baselines

Table 6 reports the full offline test and OOD results. DirectAct obtains 0.498 utility on the test split, reflecting the cost of never asking. AskAlways and raw ambiguity reach 0.920 by preserving task success but paying unnecessary clarification cost on all oracle-act episodes. The prompted heuristic reaches 0.938, but its ask rate remains 0.700, above the oracle rate of 0.500. ECUand the learned controller reach 0.958 on the test split and 0.975 on the OOD split, eliminating both missed and unnecessary clarification while maintaining high task success.

Model	Direct	Ask	CoT	ECU	ECU-Ask	95% CI	Ask errs
GPT-4.1-mini	0.420	0.632	0.632	0.976	0.343	[0.168, 0.559]	0.000
GPT-5.4-mini	0.380	0.868	0.864	0.976	0.107	[0.030, 0.219]	0.000
GPT-5.5	0.240	0.821	0.976	0.976	0.155	[0.035, 0.316]	0.000

Table 4: Current-model 100-episode API sweep. Stronger prompting narrows the gap, and GPT-5.5 with private reasoning matches ECUon this subset, but plain Ask-Needed remains below the utility-calibrated policy. "Ask errs" is the sum of ECUmissed and unnecessary clarification rates.

Category	Direct	Ask-Needed	ECU	ECUAsk	Oracle Ask
Context-resolved	1.000	0.993	1.000	0.000	0.000
Preference/social	0.400	0.400	0.980	0.400	0.400
Equivalent-outcome	1.000	0.920	1.000	0.000	0.000
Referential	-0.100	0.857	0.950	1.000	1.000
Risk-sensitive	-0.200	-0.008	0.950	1.000	1.000

Table 5: API net utility by category. ECUasks when clarification matters and avoids asking when context or outcome equivalence resolves the instruction.

The ambiguity-only diagnostic isolates the paper's central claim. All 400 canonical test episodes have multiple candidate interpretations, yet exactly 200 are oracle-act cases and 200 are oracle-ask cases. A surface-ambiguity policy therefore asks on every episode and reaches 0.920 utility with unnecessary clarification rate 1.000. A supervised uncertainty-only controller trained only on candidate count, top prior, and entropy reaches 0.900 utility, but still misses 7.0% of oracle-ask cases and asks unnecessarily in 40.0% of oracle-act cases. ECUand the full learned controller reach 0.958 utility on the same split. The learned controller is interpretable: its largest positive weights are expected-utility margin, risk, and entropy, while context resolution, equivalence, and ask cost push toward acting.

A situated contrast diagnostic makes the same point at the slice level. Among two-candidate `bring` episodes, all 80 context-resolved cases are oracle-act while all 80 referential cases are oracle-ask. For `put_away` preference episodes, visible ownership gives oracle-act on 40/40 cases, while hidden ownership gives oracle-ask on 40/40. High-entropy equivalent-outcome cases are oracle-act because all candidates share one success class, whereas high-top-prior risk-sensitive cases are oracle-ask because the wrong-action cost is high.

5.4 What does ECUbuy?

A cached decision ablation isolates the role of the equivalence guard. Replaying the same GPT-4.1-mini candidate interpretations, accepting every model-declared equivalence flag drops utility from 0.976 to 0.745 and misses 37.5% of oracle-ask cases. Never collapsing equivalent candidates keeps success at 100% but raises unnecessary clarification to 38.5%. The conservative guard is therefore needed in both directions: it prevents unsafe over-collapsing in true ambiguity while still allowing immediate action in equivalent-outcome cases.

Utility-margin calibration further explains the gap. On the main API set, ECUasks on 0/40 act-preferred episodes and 48/48 ask-preferred episodes. Prompted Ask-Needed asks on 17/40 act-preferred episodes and only 20/48 ask-preferred episodes. On the paraphrase/style stress set, ECUasks on 0/20 act-preferred and 23/23 ask-preferred episodes, while Ask-Needed asks on 7/20 and 12/23. The cached API-side margin diagnostic agrees with the oracle ask label on 99/100 main API rows, and the final ask/oracle agreement is 100/100 after the conservative context and equivalence guards. Thus the prompted baseline is not calibrated to value of information: it asks at similar rates when asking is harmful and when asking is useful.

A candidate-probability diagnostic shows what the model contributes to ECU. On the GPT-4.1-mini main API rows, the model's top success class matches the benchmark top-

Split	Method	N	Utility	Success	Ask	Unnec.
Test	DirectAct	400	0.498	0.792	0.000	0.000
Test	AskAlways	400	0.920	1.000	1.000	1.000
Test	Prompted heuristic	400	0.938	0.988	0.700	0.400
Test	ECU	400	0.958	0.988	0.500	0.000
Test	Learned controller	400	0.958	0.988	0.500	0.000
OOD	Prompted heuristic	200	0.955	1.000	0.700	0.400
OOD	ECU	200	0.975	1.000	0.500	0.000
OOD	Learned controller	200	0.975	1.000	0.500	0.000

Table 6: Offline diagnostic evaluation. Offline policies use benchmark features and are intended as transparent controllers and upper-bound diagnostics, not direct replacements for API agents.

prior success class on 97/100 episodes, with mean total-variation distance 0.057 from the benchmark success-class priors. This does not mean the model perfectly predicts the hidden intent: the top class matches the sampled hidden success class on 77/100 episodes, and referential cases remain intrinsically uncertain. What matters for the ask-or-act decision is that the model-derived utility margin tracks the oracle margin well (Pearson 0.948, Spearman 0.741 on GPT-4.1-mini; Pearson 0.991, Spearman 0.954 on GPT-5.5). This supports a bounded mechanism claim: ECUneeds plausible candidate sets and rough relative probabilities, not perfect hidden-intent calibration.

5.5 Robustness and stress tests

We first test whether the current-model result depends on the original JSON scene serialization. Reversing the object order for the same 100 GPT-5.4-mini episodes leaves API ECUunchanged at 0.976 net utility, 100% success, and zero missed or unnecessary clarifications. The ECUask/act decision changes on 0/100 shared episodes. Prompted Ask-Needed changes ask/act decisions on 10% of episodes and reaches 0.908 utility; CoT Ask-Needed changes 7% and reaches 0.926. We then replace the JSON scene with a compact natural-language scene description. API ECUremains at 0.975 utility with 100% success, zero missed clarifications, and one unnecessary clarification; its ask/act decision changes on 1/100 episodes. Prompted Ask-Needed reaches 0.788 utility and changes ask/act decisions on 8% of episodes, while CoT Ask-Needed reaches 0.904 and changes 10%. Thus the ECUresult is not explained by the original object ordering or JSON syntax, although these are still only two bounded serialization perturbations rather than a broad natural-language robustness study.

On the 50-episode API stress set, API ECUreaches 0.977 net utility and 100% success. Prompted Ask-Needed reaches 0.814 utility, misses 47.8% of oracle clarification cases, and asks unnecessarily in 25.9% of oracle-act cases. DirectAct reaches 0.320 utility. The paired ECU–Ask-Needed difference is +0.163 with 95% CI [0.040, 0.321]. This small stress test does not prove broad linguistic robustness, but it checks that the main ask/act calibration result is not limited to the original instruction and answer templates.

In the auxiliary 25-episode GPT-4.1-nano check, ECUreaches 0.722 utility versus 0.098 for Ask-Needed and 0.040 for DirectAct, with paired ECU–Ask-Needed difference +0.624 [0.074, 1.254]. This supports the direction of the result under a weaker model, but it remains too small to substitute for a broad model sweep.

We also evaluate category shift without additional API calls. Train/dev/test episodes contain only referential, context-resolved, and equivalent-outcome cases; the held-out split contains risk-sensitive and preference/social cases. ECUtransfers with 0.962 held-out utility, nearly unchanged from 0.963 on the seen-category test split. The tuned threshold and learned controller reach 0.950 but ask on all held-out episodes, over-clarifying visible preference/social cases. This is a boundary on the learning claim: the controller is transparent and effective when diagnostic categories are represented, while the expected-utility rule is more stable under this category shift.

Failure mode	Observed baseline behavior	ECUdecision reason
Referential guessing	"Bring me the black box": DirectAct chose one matching box without asking and failed.	Candidate success classes differ, so asking has higher expected utility.
Risk blindness	"Delete the old draft": Ask-Needed acted immediately; lucky success still violates the ask oracle.	High wrong-action cost makes clarification valuable even when one target seems likely.
Preference blindness	"Put my box away": the model acted with ownership hidden and selected the wrong box.	Hidden owner information prevents resolving "my" from the scene alone.
Equivalence blindness	"Move a spare folder to the table": Ask-Needed asked which spare folder, paying an unnecessary cost.	All spare folders share the same success class, so acting is sufficient.
Context overask	"Bring me the folder": Ask-Needed asked even though the active workspace identified the target.	Context-resolved salience makes immediate action utility-dominant.

Table 7: Representative first-turn calibration failures. The examples illustrate why ambiguity detection alone is insufficient: the right decision depends on outcome equivalence, context, and stakes.

The CLAMBER external sanity check is weaker and is used only as a motivation check: a direct ambiguity detector has 0.284 recall and 0.716 missed-clarification rate under our utility labels, but CLAMBER is not a situated task-utility result.

Finally, re-scoring fixed cached API outputs under alternate costs shows that the main result is not tied to one narrow reward parameterization. When ask cost varies from 0.01 to 0.35, ECUretains a positive paired utility delta over Ask-Needed from +0.201 to +0.239. When wrong-action cost varies from 0.2 to 3.0, the delta ranges from +0.138 to +0.475, with all paired bootstrap lower bounds above zero and minimum +0.070. This fixed-output analysis does not test adaptive retuning under new costs.

6 Qualitative Analysis

We label a failure event whenever the final action fails or the first-turn ask/act decision disagrees with the utility oracle. This counts successful but unnecessary questions and lucky unsafe guesses as calibration failures. On the main API set plus the private-reasoning baseline, DirectAct has 48 failure events, Ask-Needed has 45, and Ask-Needed with private reasoning has 47. In plain-text terms, API ECU has 0 failure events. The largest failure modes are risk blindness (57 events), equivalence blindness (33), and referential guessing (25).

Prompted Ask-Needed exhibits two recurring failure modes. First, it over-clarifies equivalent-outcome instructions. For example, when asked to move a spare folder to the table, it often asks which spare folder the user means even though any spare folder succeeds. Second, it under-clarifies high-risk actions. In several delete-file cases, the prompt baseline acts immediately despite high wrong-action cost. ECUavoids these failures because its decision rule explicitly compares the value of asking to the value of acting.

The analysis also clarifies what ECUis *not* doing. It is not simply asking more: its ask rate is lower than the prompted heuristic offline and close to the oracle rate in the API setting. It is not only improving second-turn grounding: conditional on asking, Ask-Needed and ECUboth ground the simulated answer successfully in the main set. The improvement comes from first-turn timing—asking when information changes the optimal action and acting when it does not.

7 Related Work

Clarification and ambiguity. Clarification has been studied in question answering, dialogue, and language-agent settings. CLAM studies selective clarification for ambiguous questions (Kuhn et al., 2022); AmbigQA constructs answers for ambiguous open-domain questions (Min et al., 2020); and CLAMBER evaluates whether language models identify and clarify ambiguous information needs (Zhang et al., 2024). Value-of-information approaches rank clarification questions by expected utility (Rao & Daumé, 2018), and conversational

search studies ask clarifying questions to resolve underspecified user needs (Aliannejadi et al., 2019). Recent work compares human and LLM clarification behavior for referential ambiguity (Madge et al., 2025). Our benchmark differs by grounding the ask-or-act decision in deterministic task reward: an instruction can be linguistically ambiguous but utility-resolved, or linguistically likely but too risky to act on without asking.

Situated instruction following and embodied language. Situated language understanding links utterances to perception, affordances, and actions (Bisk et al., 2020; Min et al., 2024). Interactive and embodied tasks such as GuessWhat?!, CoDraw, TEACh, and SayCan evaluate language in goal-directed environments (de Vries et al., 2017; Kim et al., 2019; Padmakumar et al., 2022; Ahn et al., 2022). These settings emphasize that meaning is not purely textual. CLARIFY-TO-ACTcomplements them by stripping away perception and long-horizon planning to isolate one decision: whether communication is worth its cost before acting.

Pragmatics and communicative utility. Rational Speech Acts and related pragmatic models explain communication as recursive reasoning about speakers, listeners, and utilities (Frank & Goodman, 2012; Monroe et al., 2017). Neural agents trained in referential games also show that communicative success can shape language use and interaction protocols (Lazaridou et al., 2017). We draw on this tradition but evaluate modern language agents in an ask-or-act setting where the cost of communication and the cost of wrong actions are explicit.

Social and multi-agent language agents. Recent social-agent benchmarks evaluate whether language agents can coordinate, cooperate, and reason about social contexts in multi-turn interactions (Zhou et al., 2024). CLARIFY-TO-ACTis less realistic but more controlled: rather than asking an LLM judge to score social success, the environment computes task success and clarification utility directly. This makes it suitable for low-compute, reproducible studies of pragmatic calibration.

8 Limitations

CLARIFY-TO-ACTis a synthetic text/JSON environment, not a physical simulator. This isolates ambiguity, context, equivalence, ask cost, and wrong-action cost in a controlled setting, but does not test perception, long-horizon planning, multimodal grounding, or real human interaction. The simulated user answers deterministically from the hidden intent; a released audit verifies that these answers identify the hidden success class from visible scene fields in generated oracle-ask cases and actual API asked rows, but this is not a human-response study.

The headline API evaluations use hosted models on 100-episode stratified subsets and a 50-episode paraphrase/style stress set; they are not a replacement for full-scale deployment evaluations or human user studies. We include GPT-5.4-mini and GPT-5.5 sweeps, but not open-weight local models, multimodal agents, or long-horizon embodied controllers. The offline controller is lightweight and interpretable rather than a model-weight training result. Finally, the expected-utility oracle depends on benchmark-specified priors and costs; real deployments must estimate these quantities from user preferences, domain risk, and environment state.

9 Conclusion

Clarification should be evaluated as a situated decision under uncertainty, interaction cost, and task consequences. CLARIFY-TO-ACTprovides a controlled benchmark for this decision. Across offline and API evaluations, expected communicative utility improves net task utility by asking when clarification changes the expected outcome and acting when it does not. These results suggest that pragmatic clarification is best studied as utility-calibrated interaction policy rather than ambiguity detection alone.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jodilyn Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, and Mengyuan Yan. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. URL https://arxiv.org/abs/2204.01691.

Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. Asking clarifying questions in open-domain information-seeking conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 475–484, 2019. doi: 10.1145/3331184.3331265.

Yonatan Bisk, Ari Holtzman, Jesse Thomason, Jacob Andreas, Yoshua Bengio, Joyce Chai, Mirella Lapata, Angeliki Lazaridou, Jonathan May, Aleksandr Nisnevich, Nicolas Pinto, and Joseph Turian. Experience grounds language. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 8718–8735, 2020. doi: 10.18653/v1/2020.emnlp-main.703.

Herbert H. Clark and Deanna Wilkes-Gibbs. Referring as a collaborative process. *Cognition*, 22(1):1–39, 1986. doi: 10.1016/0010-0277(86)90010-7.

Harm de Vries, Florian Strub, Sarath Chandar, Olivier Pietquin, Hugo Larochelle, and Aaron Courville. Guesswhat?! visual object discovery through multi-modal dialogue. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5503–5512, 2017.

Michael C. Frank and Noah D. Goodman. Predicting pragmatic reasoning in language games. *Science*, 336(6084):998, 2012. doi: 10.1126/science.1218633.

H. Paul Grice. Logic and conversation. In Peter Cole and Jerry L. Morgan (eds.), *Syntax and Semantics, Volume 3: Speech Acts*, pp. 41–58. Academic Press, 1975.

Jin-Hwa Kim, Nikita Kitaev, Xinlei Chen, Marcus Rohrbach, Yuandong Tian, Dhruv Batra, and Devi Parikh. Codraw: Collaborative drawing as a testbed for grounded goal-driven communication. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 6495–6513, 2019. doi: 10.18653/v1/P19-1651.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. CLAM: Selective clarification for ambiguous questions with generative language models. *arXiv preprint arXiv:2212.07769*, 2022. doi: 10.48550/arXiv.2212.07769. URL https://arxiv.org/abs/2212.07769.

Angeliki Lazaridou, Alexander Peysakhovich, and Marco Baroni. Multi-agent cooperation and the emergence of natural language. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1612.07182.

Chris Madge, Matthew Purver, and Massimo Poesio. Referential ambiguity and clarification requests: Comparing human and LLM behaviour. In *Proceedings of the Eighth Workshop on Computational Models of Reference, Anaphora and Coreference*, pp. 1–11, 2025. doi: 10.18653/v1/2025.crac-1.1. URL https://aclanthology.org/2025.crac-1.1/.

Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 5783–5797, 2020. doi: 10.18653/v1/2020.emnlp-main.466.

So Yeon Min, Xavi Puig, Devendra Singh Chaplot, Tsung-Yen Yang, Akshara Rai, Priyam Parashar, Ruslan Salakhutdinov, Yonatan Bisk, and Roozbeh Mottaghi. Situated instruction following. *arXiv preprint arXiv:2407.12061*, 2024. doi: 10.48550/arXiv.2407.12061. URL https://arxiv.org/abs/2407.12061.

Will Monroe, Robert X. D. Hawkins, Noah D. Goodman, and Christopher Potts. Colors in context: A pragmatic neural model for grounded language understanding. *Transactions of the Association for Computational Linguistics*, 5:325–338, 2017. doi: 10.1162/tacl_a_00064.

Aishwarya Padmakumar, Jesse Thomason, Ayush Shrivastava, Patrick Lange, Anjali Narayan-Chen, Spandana Gella, Robinson Piramuthu, Gokhan Tur, and Dilek Hakkani-Tur. TEACh: Task-driven embodied agents that chat. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 2017–2025, 2022.

Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Zhong Zhang, Jie Zhou, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Tell me more! towards implicit user intention understanding of language model driven agents. *arXiv preprint arXiv:2402.09205*, 2024. doi: 10.48550/arXiv.2402.09205. URL https://arxiv.org/abs/2402.09205.

Sudha Rao and Hal Daumé, III. Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information. *arXiv preprint arXiv:1805.04655*, 2018. URL https://arxiv.org/abs/1805.04655.

Tong Zhang, Peixin Qin, Yang Deng, Chen Huang, Wenqiang Lei, Junhong Liu, Dingnan Jin, Hongru Liang, and Tat-Seng Chua. CLAMBER: A benchmark of identifying and clarifying ambiguous information needs in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 10746–10766, 2024. doi: 10.18653/v1/2024.acl-long.578. URL https://aclanthology.org/2024.acl-long.578/.

Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. SOTOPIA: Interactive evaluation for social intelligence in language agents. *arXiv preprint arXiv:2310.11667*, 2024. URL https://arxiv.org/abs/2310.11667.